

AutoChip Tutorial Assignment Report

ECE - GY 9953 Advanced Project

Abstract

This report documents the completion of the AutoChip tutorial assignment, which explores LLM-assisted hardware design through iterative generation and verification of synthesizable Verilog code. AutoChip extends the ChipChat approach by implementing a structured trajectory-based refinement process with multiple candidate generation and feedback-driven iteration. The assignment comprised three main components: solving two ChipChat examples using the AutoChip framework (Part I(a)), documenting complete generation trajectories with configuration details, and developing a manual RTL design for comparison (Part I(b)). For Part I(a), two distinct hardware designs were successfully implemented and verified using AutoChip: a Binary-to-BCD Converter (combinational logic) and an 8-bit Shift Register (sequential logic). Both designs were generated using carefully engineered prompts, compiled with iverilog, and validated against provided testbenches. The Binary-to-BCD converter succeeded in 1 iteration, while the Shift Register required 2 iterations due to timing and initialization complexities. Part I(b) focused on manual RTL design of the Binary-to-BCD converter, demonstrating hand-written Verilog implementation with comprehensive documentation, alternative approach analysis, and testbench verification. This manual design provided a baseline for comparing AutoChip-generated code quality and development efficiency. The complete trajectory documentation revealed key insights into LLM-assisted hardware design: prompt engineering significantly impacts first-iteration success rates, sequential designs require more detailed specifications than combinational logic, and providing implementation templates dramatically improves complex design outcomes. All designs successfully passed their respective testbenches, demonstrating the viability of AutoChip for rapid RTL prototyping when combined with rigorous verification and iterative refinement.

1. Introduction

The AutoChip tutorial assignment explores an advanced approach to LLM-assisted hardware design that extends beyond single-shot code generation to implement a structured, iterative refinement process. Unlike traditional ChipChat approaches that rely on a single prompt-response cycle, AutoChip introduces multiple candidate generation, automated testing, and feedback-driven iteration to improve success rates and design quality.

1.1 AutoChip Framework Overview

AutoChip represents a significant evolution in LLM-assisted hardware design methodology. Traditional approaches suffer from several limitations: single attempts may fail due to subtle specification misunderstandings, no mechanism exists for incorporating compilation or simulation feedback, and designers must manually iterate through multiple prompts. AutoChip addresses these challenges through:

This methodology mirrors human design processes: initial implementation, testing against specifications, debugging based on failures, and iterative refinement until correctness is achieved.

1.2 Assignment Structure and Objectives

This assignment was structured to provide comprehensive experience with three critical aspects of AutoChip-assisted hardware design:

- Generate synthesizable Verilog using AutoChip's iterative framework
- Implement one combinational logic design (Binary-to-BCD Converter)
- Implement one sequential logic design (8-bit Shift Register)

- Document complete configuration, prompts, and trajectories
- Validate correctness through testbench-driven simulation
- Design one example by hand for comparison
- Provide comprehensive design rationale and explanation
- Analyze alternative implementation approaches
- Verify against the same testbench used for AutoChip
- Clean, self-contained notebooks
- Complete configuration documentation
- Trajectory explanations with key insights
- Professional report quality

1.3 Significance of LLM-Assisted Hardware Design

Traditional hardware design requires extensive knowledge of Hardware Description Languages (HDLs), careful attention to synthesizability constraints, timing considerations, and design-for-test principles. The RTL development phase, while critical, can be time-consuming and error-prone.

AutoChip's approach offers several potential advantages:

However, this approach also introduces challenges: ensuring LLM-generated code is synthesizable, validating functional correctness, maintaining documentation quality, and understanding generated implementations. This assignment addresses these challenges through rigorous verification and comparative analysis with manual design.

2. Tools and Methodology

2.1 Development Environment

2.2 Hardware Design and Verification Tools

- Open-source Verilog compiler for RTL compilation and elaboration
- Used for syntax checking and design elaboration
- Icarus Verilog runtime engine for executing compiled simulations
- Generates testbench output for pass/fail determination
- Produces VCD waveform dumps for optional debugging
- Waveform viewer for analyzing signal transitions
- Useful for debugging timing-related issues in sequential designs

2.3 LLM API Configuration

AutoChip was configured to use OpenAI's GPT-4o-mini model through the official Python API client.

2.4 AutoChip Framework Implementation

```
def verilog_loop(design_prompt, module_name, testbench_file,
                 max_iterations, model, work_dir=".",
num_candidates=5):
    """
    AutoChip main loop with trajectory tracking.
    Args:
        design_prompt: Natural language specification
        module_name: Expected Verilog module name
        testbench_file: Testbench filename for verification
        max_iterations: Maximum refinement cycles
        model: LLM model interface instance
        work_dir: Working directory for file I/O
        num_candidates: Designs per iteration
    Returns:
        (success, final_code, trajectory)
    """
```

- Submit current prompt (initial or with feedback) to LLM API
- Generate N candidate responses (N=5 by default)